ConnectLocal

System and backend-lead presentation guide

A practical explanation of how the Flutter apps, Express API, Supabase data platform and PayFast fit together - plus clear answers on security, scaling and known limitations.

One-line system pitch
ConnectLocal is a South African local-services marketplace. Flutter clients call a versioned HTTPS API; a modular Express backend applies identity, ownership and marketplace rules; Supabase provides authentication, PostgreSQL/PostGIS and file storage; PayFast handles hosted card checkout and verified payment notifications.

What this guide is for

- Explain the real request and data flows, not just the intended architecture in older design documents.
- Prepare accurate answers on the booking lifecycle, payment trust model, data protection and scaling.
- Separate features that are implemented from hosting assumptions, sandbox behavior and future work.

Prepared: 8 November 2026
Audience: project presentation / backend lead
Evidence: repository source, Supabase migrations, CI configuration and the test report dated 7 November 2026.

Important presentation discipline: say “the code supports / the test report records” where appropriate. The repository does not contain a production load-balancer or autoscaling configuration, and sandbox payout simulation is not a real bank transfer.

1. System at a glance

ConnectLocal is a layered modular monolith: one Express application contains separate business modules rather than independently deployed microservices. The same API serves the customer/provider app and the separate admin web app.

Flutter customer / provider app Android, iOS, Web > **HTTPS + JSON /api/v1** > **Express API routes, middleware, services** > **Supabase Auth + Postgres/PostGIS**

Other connections

Connection	Purpose	Trust boundary
API -> Supabase Auth	Sign-up, login, refresh, token validation	Supabase issues and validates identity tokens.
API -> Supabase Postgres	Profiles, providers, bookings, payments, messages and other records	Backend uses the server-only service-role key; it bypasses RLS, so API authorization is mandatory.
Flutter -> Supabase Storage (signed URL)	Upload documents/photos after asking API for a signed upload URL	The client gets a scoped short-lived upload permission, not a Supabase admin key.
API <-> PayFast	Build hosted checkout and receive ITN callback	Only a verified server-to-server notification changes payment state.
Flutter -> map tiles; API -> Nominatim	Display maps and look up/reverse-look up addresses	Map tiles render in the client; address search is through API and OpenStreetMap Nominatim.

Source-code reality: Flutter does not connect directly to the database for application reads/writes. Its normal data path is API-only. Storage file bytes are the deliberate exception: they go from the app to Supabase Storage using a signed URL issued by the API.

Stack: Flutter/Dart; Node.js 22+ and Express (ES modules); Zod validation; Supabase JavaScript client; managed PostgreSQL with PostGIS; PayFast; GitHub Actions for checks/builds.

2. How the frontend connects to the backend

Both Flutter entry points create the same ApiClient and repository: **lib/main.dart** is the customer/provider app; **lib/main_admin.dart** is the admin web application. The build receives the API origin through **--dart-define=API_BASE_URL=...**; the client normalizes it by adding **/api/v1**.

Step	What happens
1. Configure	Set API_BASE_URL at Flutter run/build time. If missing, the app deliberately shows a “not connected” screen.
2. Authenticate	Login/register goes to /api/v1/auth. The backend returns Supabase access + refresh tokens after successful authentication.
3. Call API	Repository methods call REST endpoints with JSON; authenticated requests send Authorization: Bearer .
4. Refresh	ApiClient refreshes near expiry, shares a concurrent refresh call, retries one API 401 once. Only GET calls receive automatic transient/network retries.
5. Update UI	AppState consumes API responses and notifies Flutter screens. Business state is server-authoritative; this is not an offline-first app.

Examples you can mention

- Provider search reads verified listings and active services from the API; nearby search uses server-side geospatial queries, while map tiles are rendered on the client.
- Bookings, messages, notifications and payments are read/written through ConnectLocalRepository -> ApiClient -> Express.
- Documents and photos: app asks /uploads/sign, then PUTs the bytes directly to the returned Supabase Storage URL. It submits the resulting path to the API, which checks ownership and file existence.
- Signed-in updates are polled from API endpoints every 30 seconds for bookings, conversations and notifications. There is no Supabase Realtime subscription in the Flutter/backend source inspected.

Security caveat to disclose: the current TokenStore uses shared_preferences. On web this is browser localStorage, which is readable by JavaScript in the page; the source itself marks this as a security TODO. Plan secure mobile storage and a carefully designed web session strategy (for example Secure, HttpOnly, SameSite cookies plus CSRF protection) before production.

Useful evidence: frontend/connect-local/lib/core/api_client.dart, lib/core/connect_local_repository.dart, lib/features/shared/app_config.dart, lib/main.dart and lib/main_admin.dart.

3. Backend request path and module boundaries

The API is a modular monolith organized around business capabilities. HTTP controllers are colocated with Express route files; services hold authorization/business rules; repositories centralize Supabase database access.

HTTP request /api/v1/... > **Express route + Zod schema** > **Auth / role / rate limit middleware** > **Domain service business rules** > **Repository Supabase query** > **JSON response central erro**

Module	Responsibilities (examples)
auth, users	Registration and login, profile, email/phone verification, session refresh and account actions.
providers, search, geo, uploads	Provider applications, business/service listings, availability, radius/text search, address lookup, signed uploads.
bookings	Booking creation, scheduling, accept/decline/cancel/reschedule, arrival PIN, completion.
payments	PayFast intent, checkout link, ITN verification, receipts, refunds and cash-balance records.
payouts, notifications	Encrypted payout details, payout processing, in-app notifications and optional device/push integration.
messages, reviews, admin	Conversation access, review eligibility/moderation, admin verification, reports, analytics and disputes.

Cross-cutting behavior

- verifyToken asks Supabase Auth to validate the access token, then loads public.users and rejects inactive/deleted profiles. The role comes from that database profile, not client-supplied request data.
- Routes require roles and services enforce resource ownership/booking-party rules. Zod schemas reject malformed/unexpected payload fields where strict schemas are used.
- A centralized error handler maps validation/database errors into HTTP status + JSON envelopes; Pino logs structured request/error data.
- CORS allows configured browser origins; it is a browser policy, not authentication. Requests without Origin (mobile clients, curl, webhook) still rely on route authorization or webhook validation.

API prefix: /api/v1. Health checks: /health (process) and /health/db (database connectivity).

4. Database and persistence

Supabase provides the managed PostgreSQL database and Supabase Auth identity store. The app profile lives in public.users and relates to the auth.users identity. Application repositories use supabase-js over HTTPS; versioned schema changes live in supabase/migrations.

Data area	Representative tables / relationships
Identity and business	auth.users -> public.users -> providers -> services -> service_packages. service_categories classify provider services.
Schedule and booking	provider_availability, provider_time_off; bookings connect customer, provider and service; booking_status_history records status changes.
Money	payments -> receipts; payouts relate to booking/provider; provider_payout_details stores encrypted account number + last four digits.
Communication and trust	messages/conversations, notifications, reviews, disputes, phone_otps, device_tokens.
Files and discovery	provider_documents store private Storage paths; provider_gallery/service_images store public photo references; favorites and search/audit tables support app features.

Data/design notes

- UUID identifiers and foreign keys connect users, providers, services, bookings and payment records. Database types/check constraints encode states and amounts.
- PostGIS geography points plus a GiST index support location/radius search; the client supplies search coordinates and map tiles.
- A booking stores pricing snapshots (pricing type, unit price, package name, quantity, total) so later service edits do not rewrite historical agreement details.
- Booking fee is 15% and balance is total minus fee. Commission is 10%; payout formula deducts commission and any cash already collected.
- Database migrations are run with Supabase CLI tooling. In Codespaces, direct database connectivity must use the Supabase Session pooler because the direct DB hostname is IPv6-only there.

Critical distinction: migrations enable RLS and policies are defense-in-depth for direct anon/authenticated access. The API repositories use a service-role key that bypasses RLS. For normal app API calls, the backend authorization checks are the effective control.

Schema evidence: supabase/migrations/20260807224353_connectlocal_full_schema_v2.sql and later migrations, especially 20261104230000_booking_flow_pricing_payouts_otp.sql and 20261106120000_payment_idempotency.sql.

5. Booking lifecycle: what the rules enforce

The customer requests and pays a booking fee, the provider accepts, and the customer confirms arrival with a PIN. The API owns state transitions and pricing decisions.

Customer verifies email + phone > **Books verified provider and service** > **Pays 15% booking fee by card**

Provider accepts > **Customer shares PIN on arrival** > **Job completes** > **Balance paid; review follows**

Stage	Backend checks / resulting state
Create request	Customer only; email and phone must be verified; provider must be active/verified; service belongs to provider and is active; visit/house-call option must be offered. Price is computed server-side.
Schedule	Times checked against provider working hours in SAST, selected duration/slot length and existing bookings. A 30-minute buffer is required around jobs.
Fee + provider decision	15% card deposit is paid via PayFast before provider can accept. Provider must have payout details before accepting. A declined/pending-cancelled booking triggers a booking-fee refund path.
Arrival and completion	After acceptance, customer can retrieve a 4-digit PIN. Only its HMAC hash is stored; provider submits it to start job. Up to five attempts. Completion requires an in-progress booking.
Settlement	After completion, customer pays the remaining balance by card or chooses cash; provider confirms cash receipt. Review becomes eligible after payment.

PIN and phone code safeguards

Phone OTP: cryptographically generated six-digit code, HMAC-SHA256 with server secret, five-minute expiry, five attempts, single use; hourly request limit. No paid SMS channel is integrated for production; development can echo the code to the app.

Arrival PIN: derived from booking ID and server secret, stored only as a keyed hash; attempt count is conditionally updated before comparison to reduce concurrent guessing.

Honest slot-conflict caveat: service checks occur at booking, payment initiation and provider acceptance, but the project test report still lists a simultaneous same-slot booking race and recommends a database exclusion constraint. Describe current checks as application-level conflict prevention, not a complete DB-enforced guarantee.

Time rules: stored as timestamps, evaluated in SAST (UTC+2); default schedule Mon-Fri 08:00-17:00, Saturday 09:00-13:00, until provider configures own hours.

6. Payments, idempotency and payouts

The backend creates the PayFast checkout; card numbers are entered on PayFast-hosted pages. The Flutter app opens a short-lived signed checkout link and polls the API for status. A browser return/thank-you page is not proof of payment.

Control	How it works
Server-calculated amount	The service calculates booking fee/balance from stored booking data; client does not submit a trusted amount. Currency is ZAR.
Idempotency	Client supplies Idempotency-Key. Unique database index on the key and one-open-payment-per-booking/purpose index prevent duplicate intents. Retries reuse existing payment.
ITN verification	Backend checks merchant ID, signature, PayFast source IP and PayFast server validation; then compares callback amount against saved payment.
Atomic application	Database function locks the payment row and updates payment, receipt and booking payment state in one transaction. Duplicate ITNs are recognized; one receipt per payment.
Recovery	PayFast ITN handler returns retryable 500 on infrastructure errors; duplicate/invalid messages are handled safely. Late/double card charges are sent through refund logic.

Payout reality

- Payout records calculate gross job price minus 10% ConnectLocal commission and any cash balance already collected.
- Bank account number is encrypted at rest by the application using AES-256-GCM; API responses expose only last four digits. Provider must supply details before accepting.
- Important: no bank-transfer provider is connected. With PayFast sandbox, the current payout code can mark a simulated payout as paid with a SIM reference. With live PayFast, simulated payouts are disabled and payout mode defaults off. Do not describe this as real automatic bank settlement.

Best short answer: “The customer return page is just UX. Only a verified PayFast ITN, checked against our own amount and applied atomically in PostgreSQL, changes the payment to paid.”

PayFast checkout signatures follow PayFast integration rules (MD5 signature with passphrase); this is not used for password storage. Passwords are handled by Supabase Auth.

7. Security: implemented controls and boundaries

Area	Implemented in code / migrations	How to explain it
Authentication	Supabase Auth, access/refresh tokens; token verification via Supabase; active account/profile check.	Supabase proves identity; our API rechecks local account status.
Authorization	Role-based middleware and service-level ownership/booking-party checks. Admin router applies admin requirement globally.	Role alone is not enough: each operation also checks who owns the resource.
Secrets and keys	Supabase keys required and compared at startup; code/PIN secret and bank encryption key fail fast. Secrets are environment configuration, not Flutter build defines.	Service role, PayFast credentials and encryption secret stay backend-side.
Data and files	RLS enabled; private provider-documents bucket; short-lived admin view links; signed uploads under user-specific path; type/size checks.	RLS is a second line only for API calls, because service role bypasses it.
Payment integrity	Hosted PayFast checkout, verified ITN, amount check, idempotent writes and database transaction.	We never trust the redirect or client to confirm a card payment.
Abuse controls	Zod validation; request body limit; per-route in-memory rate limits; OTP/PIN attempt limits; exact-origin CORS for browsers.	CORS does not protect API from non-browser clients; authentication and authorization do.
Logging	Pino structured logs; Authorization/query parameters are excluded/redacted; request body is not logged by pino-http config.	Logs support troubleshooting without intentionally recording credentials/tokens.

Be precise about defense-in-depth

The backend connects to Supabase using a privileged service role, which bypasses database RLS. Therefore a missing authorization check in a service could expose data even though RLS is enabled. Current design relies on verifyToken + role/ownership checks in the API; RLS is an additional barrier for direct lower-privilege access, not a substitute for backend checks.

Web session risk: access/refresh tokens are in shared_preferences, which uses localStorage on web; JavaScript executing in the page could read them. This is a known hardening task, not a claim that web token storage is equivalent to HttpOnly cookies.

Also avoid saying “fully POPIA compliant” or “formally penetration-tested.” The test report lists privacy/consent/data-export/admin-audit work as not started and no formal penetration test is recorded.

8. Load balancing and scaling: current vs next step

Current state: the repository configures one Node/Express server process. There is no load-balancer, replica count, autoscaling policy or queue configuration checked into the project. A host or Codespaces tunnel may provide network ingress, but do not claim the application is currently load-balanced based on this code.

What can scale relatively cleanly

- HTTP request state is primarily in Supabase and bearer JWTs, so the API is mostly stateless and can be placed behind a managed HTTPS load balancer without sticky sessions.
- Supabase is a managed Postgres/Auth/Storage service. Search and table indexes support primary reads; paginated API paths constrain many lists.
- The database payment function and unique indexes coordinate idempotent payment processing across competing requests.

What must change before adding API replicas

Current design detail	Scaling concern	Production-ready direction
Rate limits stored in process-local Maps	Each replica has its own counters; requests spread across replicas can evade limits.	Use shared Redis or gateway-level distributed limits; retain tighter per-account controls.
Payout scheduler starts in each API process	Each replica runs a timer. Conditional DB status claim helps avoid duplicate payout state changes, but repeated scheduler work is wasteful and real transfer integration needs robust job ownership.	Move to a singleton worker/queue with durable retries, idempotent transfer requests and alerting.
In-memory Nominatim throttle/cache	Limits and cache are per process; replicas increase outbound calls.	Shared cache/rate limiter; keep within Nominatim usage policy or use a contracted geocoder.
No booking exclusion constraint yet	Near-simultaneous bookings may pass application checks.	Add reviewed PostgreSQL exclusion/locking design; test concurrent bookings and payment race cases.
No queue/outbox for notifications	Notifications are best-effort after domain actions.	Transactional outbox + worker for reliable async notifications and email/push.

Recommended scale path: first measure request latency/errors and DB saturation; optimize queries/indexes/pagination; deploy behind managed HTTPS ingress; externalize rate limit/cache state; move scheduled/async work into a durable worker; only then raise API replica count. Keep payment/booking correctness anchored in DB constraints and idempotent operations.

This is a sensible evolution of the modular monolith; microservices are not required for the current academic demo load.

9. Presentation questions: concise answers

Likely question	Suggested answer
How is the frontend connected to the backend?	Flutter uses a build-time API_BASE_URL, sends HTTPS JSON requests to /api/v1 with a bearer access token, and uses a repository/AppState layer to map API responses to screens.
Why use a backend if Supabase already exists?	The API centralizes business rules and ownership checks, calculates prices, manages booking/payment workflows and hides privileged Supabase and PayFast credentials. Supabase supplies managed auth, database, geospatial and storage capabilities.
Are you using microservices?	No. It is a modular monolith with separated route/service/repository modules. That keeps deployment and transactions simple for a small project while preserving capability boundaries.
How do you load balance requests today?	We do not currently configure load balancing or multiple API replicas in this repository. The API is designed mostly stateless, but in-memory rate limits and the per-process payout timer must be addressed before horizontal scaling.
How do you stop unauthorized users accessing bookings?	The API validates the Supabase token, loads the active user and DB-backed role, then checks role and booking-party/resource ownership in middleware/services before repository calls.
Does RLS stop a backend bug?	Not for the current repository path: the API uses Supabase service role, which bypasses RLS. RLS is defense-in-depth for lower-privilege direct access; application authorization remains critical.
How is card payment confirmed?	PayFast sends ITN; backend checks signature, source address, server validation and amount against the stored payment. The DB transaction applies payment + receipt + booking state; redirect alone never marks paid.
How do you prevent duplicate payments?	The client key, database unique indexes, one-open-payment rule and row-locked database function make intent and ITN processing idempotent.
Are provider payouts real?	No bank-transfer provider is integrated. Sandbox mode can simulate payout records; live mode leaves payouts off/pending until a transfer provider is connected.
Do you use realtime messaging?	The app refreshes conversations, notifications and booking data by polling the API every 30 seconds. The source does not currently use Supabase Realtime subscriptions.
What is the next major scaling/security improvement?	Shared distributed rate limiting and a durable worker/queue, plus a database booking-conflict constraint. For web security, move away from JS-readable token storage.

30-second spoken summary:
“ConnectLocal is a Flutter marketplace backed by a versioned Express API and Supabase. The API is the trust boundary: it validates Supabase identity, checks role and ownership, applies booking and pricing rules, and accesses Postgres through repositories. PayFast handles card entry; only a verified ITN updates payment state. We use a modular monolith today; the API is not yet load-balanced, and scaling needs shared rate limits and a dedicated worker.”

10. Current evidence and limitations to state openly

The project test report is dated 7 November 2026 and says its full booking smoke test used the live Supabase project. Those are reported results, not tests rerun while preparing this guide.

Evidence in TEST_REPORT.md	Reported result
Backend	Lint clean; 243 unit + 8 integration tests passed.
Flutter	Analyze clean; 57 tests passed; both customer/provider and admin web builds succeeded.
End-to-end	35/35 smoke steps passed against live DB, including signup, provider verification, fee ITN, PIN, balance and review checks.
Database	Migrations reported up to date at time of that report.
Dependency audit	Report lists an open high-severity Zod issue; CI audit is configured to fail on critical severity, not all high issues.

Known caveats from code/report

- No in-repo production load balancer/autoscaling configuration; process-local rate limits and scheduler should be reworked before replicas.
- Simultaneous booking conflict needs stronger database-level enforcement; test report recommends an exclusion constraint.
- Bank transfer is not integrated; simulated payout mode is sandbox behavior.
- Web access and refresh tokens are kept in localStorage via shared_preferences. Treat as a hardening TODO.
- Phone OTP delivery is not backed by a paid SMS gateway; development can return the OTP in API response. Production requires a real delivery channel and safe suppression of dev echo.
- POPIA work (privacy notice/consent, export/deletion processes and admin audit coverage) is listed as not started in the report.
- The test report also lists a not-yet-run real PayFast sandbox click-through and a need to confirm the deployed browser CORS allowlist.

Hosting wording: README/START describe a Codespaces demo backend and a Netlify-hosted web demo; older deployment documents describe Render. GitHub Actions in this repository run checks/tests/builds, not an actual deployment. Confirm where the current demo is hosted before naming the live deployment platform.

CI currently checks backend on Node 22/24 and Flutter analyze/tests/web builds. It is validation automation, not proof of production deployment or autoscaling.

11. Source map for follow-up questions

Use these files if an assessor asks where a behavior lives. The repository has older architecture/ERD/deployment docs that sometimes describe intended designs rather than current implementation; prioritize current source and migrations.

Topic	Repository evidence
API assembly and routes	backend/src/app.js; backend/server.js; backend/src/routes/
Token/role and validation	backend/src/middleware/auth.js; backend/src/middleware/validation.js; backend/src/middleware/rateLimit.js
Supabase clients and CORS	backend/src/config/supabase.js; backend/src/config/cors.js
Booking and scheduling	backend/src/services/bookingService.js; backend/src/utils/schedule.js; backend/src/config/business.js
PayFast and idempotency	backend/src/services/paymentService.js; backend/src/integrations/payfast.js; supabase/migrations/20261106120000_payment_idempotency.sql
Payout and encryption	backend/src/services/payoutService.js; backend/src/utils/encryption.js
Storage and uploads	backend/src/integrations/storage.js; backend/src/routes/uploads.js; supabase/migrations/20261105090000_provider_applications_storage.sql
Flutter API connection	frontend/connect-local/lib/core/api_client.dart; frontend/connect-local/lib/core/connect_local_repository.dart; frontend/connect-local/lib/features/shared/app_config.dart
Flutter state and entry points	frontend/connect-local/lib/features/community/state/app_state.dart; frontend/connect-local/lib/main.dart; frontend/connect-local/lib/main_admin.dart
Schema and RLS	supabase/migrations/20260807224353_connectlocal_full_schema_v2.sql; 20260807224408_connectlocal_rls_policies.sql; later dated migrations
Quality evidence	TEST_REPORT.md; .github/workflows/backend-ci.yml; .github/workflows/frontend-ci.yml
Project overview / run instructions	README.md; START.md; backend/README.md

Terminology

- **ITN:** PayFast Instant Transaction Notification, a server-to-server payment status callback.
- **RLS:** PostgreSQL Row-Level Security policies that filter operations by database role/user.
- **Idempotency:** repeated processing of the same logical operation returns the same result rather than creating duplicate effects.
- **Modular monolith:** one deployable application with internal module boundaries, rather than a set of independently deployed microservices.

Closing message: emphasize that ConnectLocal centralizes security-sensitive decisions in the backend, uses Postgres constraints/transactions for financial integrity, and has an explicit path to scale. Be candid that the current presentation/demo build is not yet a production-hardened multi-instance deployment.